

Fall 2021

Deep Learning in Indoor Human Activity Recognition with Millimeter Wave Radar

Jieyun Hu

Follow this and additional works at: <https://lib.dr.iastate.edu/creativecomponents>



Part of the [Digital Communications and Networking Commons](#)

Deep Learning in Indoor Human Activity Recognition with Millimeter Wave Radar

by

Jieyun Hu

A Creative Component submitted to the graduate faculty
in partial fulfillment of the requirements for the degree of
MASTER OF SCIENCE

Major: Computer Science

Program of Study Committee:
Carl K. Chang, Major Professor
Ying Cai
Simanta Mitra

The student author, whose presentation of the scholarship herein was approved by the program of study committee, is solely responsible for the content of this report. The Graduate College will ensure this report is globally accessible and will not permit alterations after a degree is conferred.

Iowa State University

Ames, Iowa

2021

Copyright © Jieyun Hu, 2021. All rights reserved.

DEDICATION

I would like to dedicate this report to my husband Lei , and to my kids Olivia and Kevin, for their love and support. I would also like to thank my advisors and labmates for their guidance and help.

TABLE OF CONTENTS

	Page
LIST OF TABLES	v
LIST OF FIGURES	vi
ACKNOWLEDGMENTS	viii
ABSTRACT	ix
CHAPTER 1. INTRODUCTION	1
CHAPTER 2. RELATED WORK	4
2.1 Data Preprocessing of radar 3D point cloud data	4
2.2 Deep Learning in Human Activities Recognition	4
CHAPTER 3. METHODS AND PROCEDURES	7
3.1 Dataset	7
3.2 Data Preprocessing	7
3.2.1 Tabularization	7
3.2.2 Sorting and Padding	8
3.2.3 Sliding Window	8
3.3 Deep Learning Neural Networks	9
3.3.1 Multiple Perceptron (MLP)	9
3.3.2 Convolutional Neural Networks (CNN)	10
3.3.3 Recurrent Neural Networks (RNN)	10
3.3.4 The Transformer	11
3.3.5 Hybrid Deep Learning Architecture	14
3.4 Equipment Setup and Data Collection	17
3.4.1 Equipment	17
3.4.2 Data collection	19
3.5 Automatic Labeling	20
3.6 Data Preprocessing, Training and Prediction	22
CHAPTER 4. RESULTS	24
4.1 Model Comparison	24
4.2 Hyperparameter Optimization	25
4.2.1 Hyperparameters Selection and Tuning	26
4.2.2 Performance of Tuned Model	26
4.3 Comparison of Data Preprocessing Methods: Voxelization and Tabularization	27

4.4 Preliminary Results on the Collected Data from Smart Home Lab	28
CHAPTER 5. SUMMARY AND DISCUSSION	33
BIBLIOGRAPHY	35

LIST OF TABLES

	Page
Table 4.1 Model Comparison	25
Table 4.2 List of Tuned Hyperparameters	26
Table 4.3 Model Comparison with Tuned Model	27
Table 4.4 Comparison of Tabularization and Voxelization	28

LIST OF FIGURES

	Page
Figure 3.1 An Overview of Data Preprocessing	8
Figure 3.2 Sliding Window Technique for Radar Sensor Data	9
Figure 3.3 Architecture of the LSTM and Structure of a LSTM Cell	11
Figure 3.4 BiLSTM Architecture	12
Figure 3.5 The Transformer Architecture	13
Figure 3.6 Scaled Dot-Product Attention	14
Figure 3.7 Multi-head Attention Consist of Several Attention Layers Running in Parallel	14
Figure 3.8 The Architecture of the Vision Transformer	15
Figure 3.9 The Architecture of CNN and the Transformer Hybrid Model	16
Figure 3.10 The Architecture of Multiple-MLP and the Transformer Hybrid Model . . .	17
Figure 3.11 The Architecture of Multiple-CNN and the Transformer Hybrid Model . . .	18
Figure 3.12 IWR8643 Packaged mmWave Sensor with DSP and MCU Evaluation Module	19
Figure 3.13 An Example of Raw Data	19
Figure 3.14 An Example of 3D Point Clouds	20
Figure 3.15 Five Collected Activities	21
Figure 3.16 Plots of Number of Non-static Points of Falls and Sits/Stands activities . .	22
Figure 3.17 An Overview of Automatic Labelling	23
Figure 4.1 Loss Graphs of Models	25
Figure 4.2 Confusion Matrices	29

Figure 4.3	Loss Graphs of Different Hyperparameters during Tuning	30
Figure 4.4	Comparison of Untuned Model and Tuned Model	31
Figure 4.5	Accuracy and Loss Comparison of Untuned Model and Tuned Model	31
Figure 4.6	Loss Graphs of CNN+Transformer and BiLSTM Models with Smart Home Lab Data	32
Figure 4.7	Confusion Matrices of CNN+Transformer and BiLSTM with Smart Home Lab Data	32

ACKNOWLEDGMENTS

I would like to thank my committee chair Dr. Carl K. Chang, and my committee members, Dr. Simanta Mitra and Dr. Ying Cai for their guidance and constant support throughout the projects.

In addition, I would like to thank all the members in the Smart Home Lab for their helpful suggestions and support to my research work. I would like to give special thanks to Mingdian Liu who gave me many technical support and suggestions on my research.

ABSTRACT

Human activity recognition (HAR) is an active field of study concerned with automatically identifying human activities by analyzing and classifying the data captured from sensors. HAR has received great attention due to its wide application in healthcare and eldercare. Deep learning models such as multilayer perceptron (MLP), convolutional neural network (CNN), and recurrent neural network (RNN) have been widely applied to HAR. Recently, Transformer emerges as a new type of deep learning neural network, which has been a breakthrough in natural language processing tasks, and is showing great promise in the field of computer vision. In our research, we experimented with three Transformer models and three non-transformer models using a public indoor millimeter wave radar dataset called MMActivity. The highest prediction accuracy of 96.10% was achieved by a Transformer model integrating CNN with the Transformer encoder, which is comparable to the prediction accuracy of 95.90% achieved by the non-transformer model BiLSTM. When applying to the dataset collected by our own millimeter wave radar, our Transformer model integrated with CNN also showed a high prediction accuracy of 93.6%.

CHAPTER 1. INTRODUCTION

Ageing is related to the decreasing ability to perform activities of daily living, affecting physical and mental health. Human activity recognition (HAR) systems are able to support the patients or elderlies, by monitoring their daily activities patterns and intervening when something abnormal occurs.

A typical HAR system includes four steps. Data collection is the first step where data is captured from a single or multiple sensors. Next, data is preprocessed such as handling missing value, data transformation and data segmentation. Then, the data is sent to train a machine learning or a deep learning model. After the model is well trained, it can be deployed to predict human activities ([Gavrilin and Khan \(2019\)](#)).

There are two types of sensors which commonly used in HAR research: wearable sensors and ambient sensors. Wearable sensors are the sensors that are embedded into smartphones, smartwatches, or other wearable devices. For example, Accelerometer, thermometer, GPS, and Gyroscope are all wearable sensors ([Fauzi et al. \(2018\)](#)). Ambient sensors such as cameras, radars, and Wi-Fi devices are the sensors which can be fixed in the environment or attached to some objects like wall, door, floor etc., Wearable sensors work in a relatively large space while ambient sensors require more complex installation and subject to environment. However, in the elderly care or the health care monitoring system, wearable sensors are always obtrusive for older users. On the other hand, ambient sensors can be deployed around the area to monitor the activities of older people or patients with no obtrusiveness.

Vision based techniques such as cameras are widely used in HAR research and outperform other sensors on the condition of a clear, frontal view of the activity. However, due to the customer's concern on intrusive method, camera have a low acceptance in domestic and commercial settings. On the other hand, radar sensors can effectively infer human activities while

reducing the privacy risks using radio frequency signal ([Çağlıyan and Gürbüç \(2015\)](#)). Moreover, radar sensors are robust to light and weather conditions such as fog and rain so it is able to be applied in various of environments.

In the past, conventional machine learning techniques such as SVM ([Rosenblatt \(1961\)](#)), Random Forest ([Schmidhuber \(2015\)](#)) were applied to HAR. However, conventional machine learning approaches usually perform feature engineering and classification independently, which require a deep understanding of the relationship between features and targets. Deep learning, a rapidly evolving technology, has emerged as a powerful tool in many areas including HAR in recent years. Deep learning models can seamlessly integrate feature computation and classification and thus they can process raw data directly. Furthermore, with the help of GPU, deep learning models can learn complex features and compute huge amount of data quickly and efficiently.

The Transformer is a deep learning model gaining popularity especially in Natural Language Processing (NLP) area such as language translation, text to speech transformation ([Vaswani et al. \(2017\)](#)). Like RNNs, Transformers could also process sequential data (e.g., a sentence), however, Transformers avoid recursion in RNNs to allow parallel computation and improve performances by self-attention, a mechanism scores the attentions between each input (e.g., each word in a sentence). Inspired by the Transformer scaling successes in NLP, researchers integrated the Transformer architecture in fields of image classification ([Dosovitskiy et al. \(2020\)](#)), image generation ([Parmar et al. \(2018\)](#)) and video action recognition ([Bilkhu et al. \(2019\)](#)), and the results showed Transformers models worked more efficient and achieved a higher accuracy than the traditional deep learning models ([Dosovitskiy et al. \(2020\)](#), [Parmar et al. \(2018\)](#), [Bilkhu et al. \(2019\)](#)).

Many deep learning architectures such as Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), Recurrent Neural Network (RNN) are widely used in HAR study ([Chen et al. \(2020\)](#)), but very few was reported using the Transformer models. The successful studies of using the Transformer in image classification and video action recognition fields inspired us to apply the Transformer architecture on our radar-based human activity recognition study. In this study,

different hybrid deep learning models with the Transformer architecture were proposed for HAR, and one of the Transformer model were further optimized with an existing mmWave point cloud dataset. Then the Transformer model was tested with our newly collected dataset, and compared with a BiLSTM model. Chapter 1 gives a brief introduction of research background and motivation. Chapter 2 discusses some related work on data preprocessing on point cloud data, deep learning models for human activity recognition, and vision and video Transformers. Chapter 3 contains (1) our approach to preprocess point cloud data, (2) the architectures of deep learning models in our research, (3) how data was collected, preprocessed in our smart home lab. (4) how the collected data was split, trained and tested. Chapter 4 shows (1) the results of different hybrid Transformer models, (2) the comparison with the traditional deep learning models, (3) the comparison of our tabularization preprocessing method with the voxelization method, (4) the experimental results on our collected dataset. Lastly, Chapter 5 concludes the work and discusses the future work.

CHAPTER 2. RELATED WORK

2.1 Data Preprocessing of radar 3D point cloud data

Data preprocessing is a step to transform the raw data in a useful and efficient format before applying to machine learning models. In this research, the raw data we wanted to process is a type of 3D point cloud data which is a set of data points in space representing an object. Point cloud data is irregular and non-uniformed, thus it needs to be transformed to a structured and uniformed representation before passing to deep learning models. The most common way is to voxelize the point cloud data meaning segment the data points into voxels. [Singh et al. \(2019\)](#) converted the point cloud data collected from a FMCW millimeter wave radar into voxels of dimensions $10 \times 32 \times 32$ with each voxel represents the number of data points within its boundary. The voxelized data was then fed the preprocessed data into deep learning models to classify human activities. [Zhao et al. \(2019\)](#) first separated data points by each subject, and then voxelized on the point cloud of each subject before using LSTM to identify people. However, there will be information loss during voxelization since multiple points within the same grid are merged, and other attributed beside x, y, z coordinates are not utilized during voxelization process.

2.2 Deep Learning in Human Activities Recognition

Deep learning is a subset of the machine learning that uses multiple layers to extract higher-level features from the raw input. In recent years, deep learning has achieved remarkable prosperity in many areas such as computer vision, natural language processing. Unlike conventional machine learning methods, deep learning uses its multiple layer structures to find optimal features and handle indirect relationships between features and goals, thus it eliminates feature engineering. Moreover, with huge data and powerful computing resources like GPUs, deep learning models are able to efficiently extract complicated features and achieve high accuracies for

inference. Many studies in HAR showed that deep learning models can successfully extract complicated features from varied sensor data and obtain high recognition accuracies.

A multilayer perceptron (MLP) is a class of feedforward neural network which consists of multiple layers of neurons, and the model is suitable for many classification and regression prediction problems ([Rosenblatt \(1961\)](#)). [Myo et al. \(2019\)](#) designed MLP classifier to classify six activities such as sitting, walking, walking upstairs using UCI-HAR dataset collected through smartphones for HAR research, and achieved overall 98.32% accuracy. A convolutional neural network (CNN) is a type of deep neural network commonly applied to analyze visual images. It is usually composed of an input layer, one or more convolutional and pooling layers, followed by one or more fully connected layers. The convolutional layers in CNNs allow the model to learn different aspects of features from the images, resulting in more accurate image classification ([Schmidhuber \(2015\)](#)). CNNs can either be performed as a classifier for radar-based HAR or a feature extractor to learn high-level features from the time range input ([Li et al. \(2019\)](#)). [Kim and Moon \(2015\)](#) proposed a CNN model to a raw micro-Doppler spectrogram for both human detection and human activity classification problem, and they achieve accuracy results of 97.6% for human detection and 90.9% for classification. A recurrent neural network (RNN) is a class of artificial neural which uses sequential data or time series data. It is mainly used in the fields of natural language processing and speech recognition ([Tealab \(2018\)](#)). RNNs are able to extract temporal and semantic information from temporal sequences so are appropriate for processing sensory data which are time series data for HAR. Standard RNNs fail to learn long-term context of sequential data because the vanishing gradient problem occurs as the time steps increased [103]. Long short term memory (LSTM) which is a type of RNNs is able to resolve the vanishing gradient problem by using memory units called gates ([Gers et al. \(2002\)](#)). [Wang et al. \(2019\)](#) introduced a stacked RNN with long short-term memory to identify human motion and achieved an average accuracy of 98.98%. In some scenarios, a model with a single neural network architecture may not be sufficient to capture the complicated features and predict the right classes. Hybrid deep learning models integrate with different neural network

architecture and take advantages of each model, thus improve the performance. For example, since CNNs captures spatial relationships while RNN capture temporal relationships, combining CNNs and RNNs tends to enhance the ability to recognize human activities that vary in time and space (Li et al. (2019)). Wang et al. (2016) applied a hybrid model combined CNN and RNN for fine-grained dynamic gesture recognition in the radiofrequency spectrum and achieved high recognition rates of 87%.

The Transformer is a type of deep learning model used widely in Natural Language Processing (NLP) such as language translation, text to speech transformation. Transformers work on sequential data just like RNNs. Moreover, Transformers use self-attention mechanism to address the associations between any two input, thus can process very long inputs. Furthermore, Transformers handle input in parallel, so that they are trained faster than RNNs. Inspired by the successful applications of the Transformers in NLP, many researches succeeded in combining CNN-like architectures with self-attention for image classification (Dosovitskiy et al. (2020)), image generation (Parmar et al. (2018)) and video action recognition (Bilkhu et al. (2019)). Dosovitskiy et al. (2020) split an image into patches and sent the sequence of the embedded patches into a Transformer encoder for image classification. This Vision Transformer generally outperformed ResNets with the same computational budget and beat the state-of-the-art deep learning models when trained on large datasets. Girdhar et al. (2019) introduced an Action Transformer for recognizing and localizing human actions in video clips. The model combined a spatiotemporal I3D architecture, a region proposal network(RPN) and Transformer encoders, and outperformed the state-of-the-art on AVA (Gu et al. (2018)) dataset by a significant margin.

In this study, the point cloud outputs are spatiotemporal data which contains both space and time information. CNN architectures can extract features from space information while the Transformer architectures can handle time series data. The goal of the research is to experiment different architectures of the Transformer on radar-based HAR and compare their performances with the traditional neural networks.

CHAPTER 3. METHODS AND PROCEDURES

3.1 Dataset

MMAActivity, a millimeter-wave radar dataset collected by TI's IWR1443BOOST radar was leveraged in our research (Singh et al. (2019)). The radar set up with four receiver and three transmitter antennas, which enable tracking multiple objects with their distance and angle information. Five activities including walking, jumping, jumping jacks, squats and boxing were performed by two users. Total duration of the data is 93 minutes. The data are in point clouds format containing spatial coordinates (x, y, z in meters) along with velocity in meters/second and intensity(dB). The sampling rate of the radar is 30 frames per second.

3.2 Data Preprocessing

Point clouds are disordered and uneven, and they are hard for deep learning models to process. To make the data structured and regular, we did three steps: 1. Tabularization. Point cloud data in each frame was converted to tabular form containing features of x, y, z coordinates, velocity, and intensity. 2. Sorting and padding. Tabular data in each frame were sorted by x coordinate and padding with 0s on both rows and columns. 3. Sliding window. Continuous frames within two seconds were reassembled to a window. Two consecutive windows have 10 frames overlapping. Figure 3.1 showed the overview of data preprocessing in our research.

3.2.1 Tabularization

The output data are saved in text files which are hard for models to handle. Thus, we parsed the text files to tabular data with columns of x, y, z coordinates, velocity, and intensity. By aggregating the data points in each timestamp, we were able to present point cloud data of each frame in a tabular format.



Figure 3.1 An Overview of Data Preprocessing

3.2.2 Sorting and Padding

In point clouds, the coordinate values show the distances between the points. The points with closed coordinates values are clustered, thus could represent the same part of the body. For example, if we sort the points by x coordinate, the first a few points may represent the left hands of the body. In this study, we sorted the point clouds data of each frame by x coordinate to endow them with some spatial information. The point clouds data from each frame are uneven, some frames have more points, while some have less. To make data uniformly, we firstly found the maximum number of points in all frames and padded the frames of fewer points with 0s. We also added an extra column with 0s making the total column number to an even number in order to facilitate data partition in the modeling process.

3.2.3 Sliding Window

Sliding window is a popular segmentation technique splitting continuous time series data into equal length of windows for model training (Banos et al. (2014)). In this study, the data was

segmented into 2 seconds windows (60 frames in total) with 10 frames overlapping to reduce possible information loss between two adjacent sub windows. Figure 3.2 demonstrated the how sliding window works in this study.

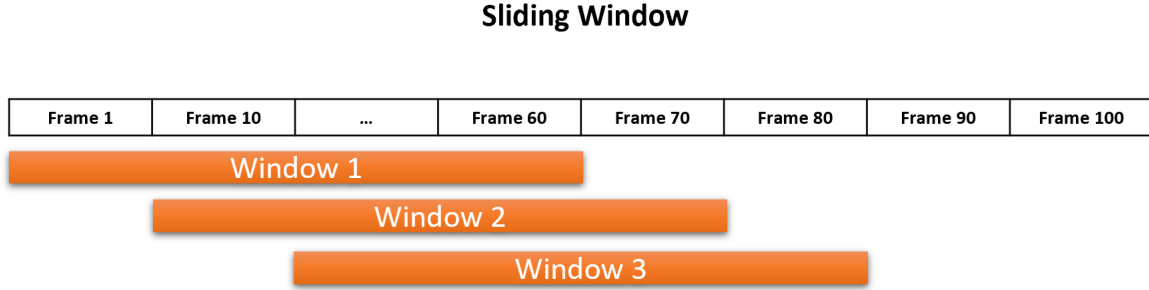


Figure 3.2 Sliding Window Technique for Radar Sensor Data

3.3 Deep Learning Neural Networks

Classical classifiers usually perform feature engineering and classification independently, which require a deep understanding of the relationship between features and targets. Deep learning models can seamlessly integrate feature computation and classification and thus they can process raw data directly.

3.3.1 Multiple Perceptron (MLP)

A MLP is a class of feedforward neural network. The MLP consists of three layers: an input layer, one or more hidden layers and an output layer. Data is fed to the input layer, then the hidden layers provide levels of abstraction, and the output layer makes a decision. MLPs are often applied to supervised machine learning problems ([Rosenblatt \(1961\)](#)). By training on the input and output paired data, MLPs are able to learn the linear or non-linear relations between those

inputs and outputs. MLPs are commonly used for tabular datasets, not for image or time-series datasets. In our research, we applied MLPs with two hidden layers extracting features from tabular data in each frame and stacking with the Transformer architecture.

3.3.2 Convolutional Neural Networks (CNN)

A convolutional neural network is a type of deep neural networks, usually applied to analyze visual images. It is usually composed of an input layer, one or more convolutional and pooling layers, followed by one or more fully connected layers ([Aljarrah and Ali \(2019\)](#)). The convolution operation is applied to either input nodes or hidden nodes of the previous layer by multiplying the input data with a filter. It was shown that CNNs outperformed the conventional machine learning algorithms such as HMM, SVM in recognition of activities using multiple accelerometer and gyroscope sensors ([Ha and Choi \(2016\)](#)). Like MLPs, CNNs can be also worked in a part of a deep learning model to perform feature extraction, but they work better in image feature extraction than MLPs. In our study, we converted point cloud data into an image like data which have fixed width, length, and channels. We used pure CNNs to train the model, but we also mixed CNNs with the Transformer architectures. In the mixed model, the meaningful features that extracted by the CNN were fed into the Transformers to perform human activity recognition. We will talk about the mixed architecture in the hybrid model section.

3.3.3 Recurrent Neural Networks (RNN)

A recurrent neural network (RNN) is a class of artificial neural networks that contain internal state (memory) to process sequences of inputs ([Schmidhuber \(2015\)](#)). It is mainly used in the fields of natural language processing and speech recognition.

Long short-term memory (LSTM) is a type of RNN, which contains memory blocks with cells to store information for long periods of time. LSTMs were developed to solve the vanishing gradient problem that can be encountered when training traditional RNNs. In the study of [Pienaar and Malekian \(2019\)](#), 94% accuracy was obtained by applying the LSTM model on

tri-axial accelerometer data for human activity recognition. Figure 3.3 shows the LSTM architecture on the left hand side and the structure of LSTM cell on the right hand side.

Bidirectional long short-term memory (BiLSTM) is another type of RNN, which contains two LSTM chains: one taking the input from the early timestamp to the late timestamp, and the other from late to early. The outputs of the two chains of BiLSTM were concatenated and applied the softmax function for classification. Aljarrah and Ali (2019) proposed a BiLSTM model for identifying 13 classes of human activities in MHEALTH dataset, and achieved the accuracy of 97.64%. Figure 3.4 illustrates the architecture of BiLSTM in our research.

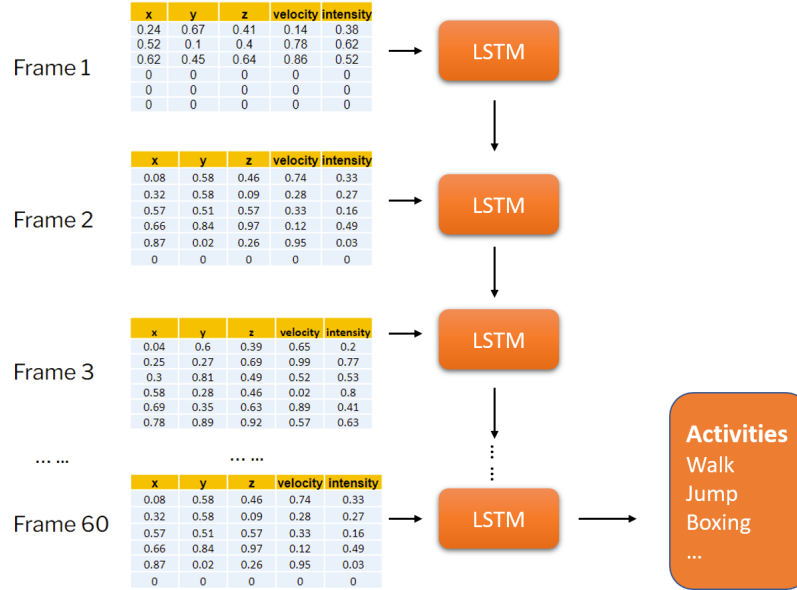


Figure 3.3 Architecture of the LSTM and Structure of a LSTM Cell

3.3.4 The Transformer

The Transformer is a type of neural network architecture that have been gaining popularity. It used widely in Natural Language Processing (NLP) such as language translation, text to speech transformation. Vaswani et al. (2017) firstly proposed the Transformer architecture which contains a stack of encoders and a stack of decoders, as seen in Figure 3.5. The function of encoders and decoders are the same with RNN Seq2seq models which used widely in NLP before

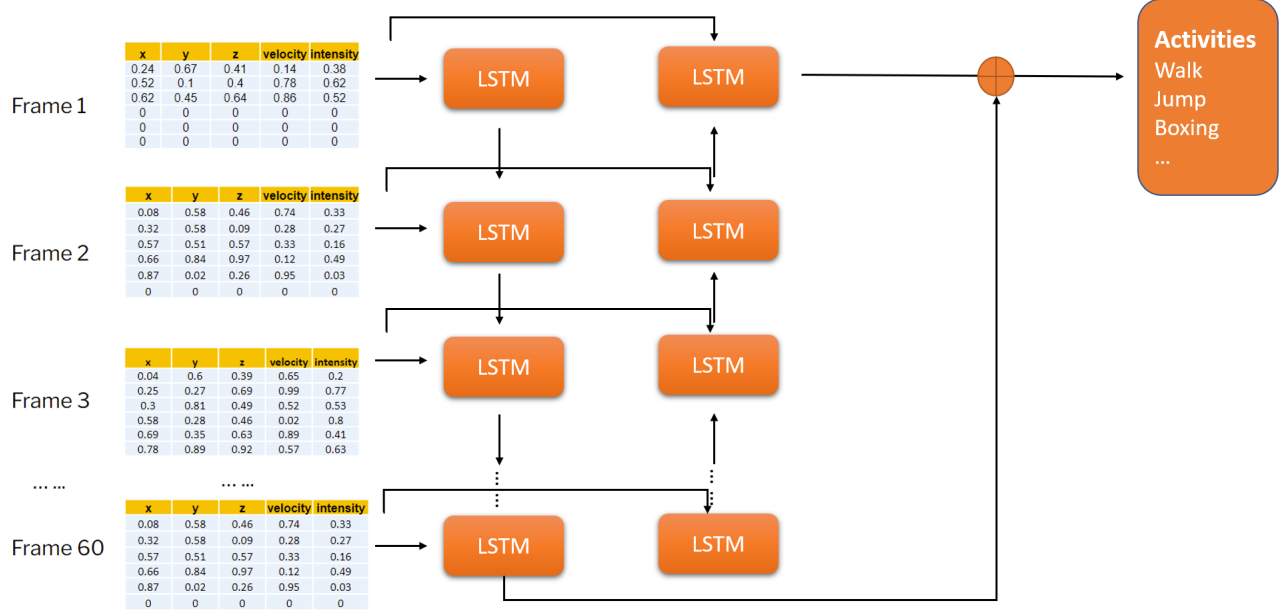


Figure 3.4 BiLSTM Architecture

the appearance of the Transformer. The encoders process each input in a sequence, like each word in a sentence, and capture the context (a vector) from the input sequence, so that the decoders can reproduce an output sequence, like translated sentence. Furthermore, the Transformer introduces the attention mechanism which looks at an input sequence and decides at each step which other parts of the sequence are important.

Figure 3.6 shows the architecture of the Attention layer in the Transformer. The equation 3.1 illustrates attention function in a mathematical way. In the attention layer, it firstly creates three vectors called Query, Key and Value from each input vector, like embedding of each word. Query is a feature vector that describes what we are looking for the sequence. Key is also a feature vector showing which element might be important. Value is a feature vector that will be averaged over. A dot product function is applied on each input in the sequence (represented by Q) with all other input (represented by K). A SoftMax function is applied to the dot product to scale the weights between 0 and 1. Those weights are then applied to all inputs (represented by V).

Multi-head attention is an extension of attention mechanism that generates multiple Query, Key

and Value for each input, so that it is able to capture multiple attentions, as seen in Figure 3.7. Overall, the attention layer allows each input to associate with other inputs in the sequence, so that when we want to infer the next output, we only need to pay attention to the important input.

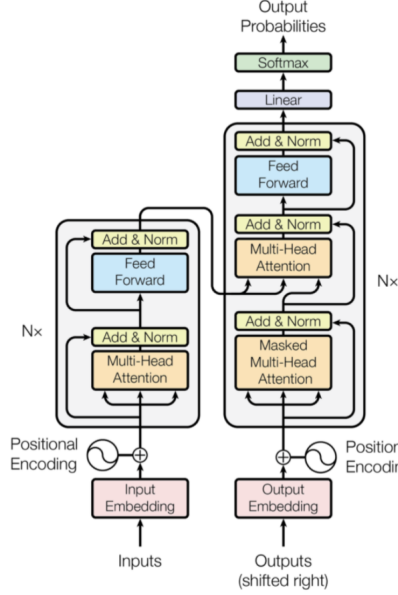


Figure 3.5 The Transformer Architecture

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (3.1)$$

When using RNN (e.g. LSTM) to process the long sequence data, the inputs need to feed into the model sequentially, thus it takes time for the signals to traverse from the early time step to the late time step, as seen in Figure 3.3. However, with the attention layer, the Transformer is able to process the whole inputs in parallel which decrease the computational time. Moreover, the Transformer does not suffer from long input sequence like RNN which may ‘forget’ the input when the sequence is too long.

In last few years, the Transformers became more and more popular in image classification area. Parmar et al. (2018) proposed a Vision Transformer (ViT) using only encoder part of the Transformer for image classification, and obtained an excellent result compared to CNNs while requiring substantially fewer computational resources to train. The architecture of the Vision

Scaled Dot-Product Attention

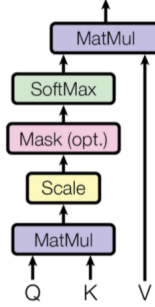


Figure 3.6 Scaled Dot-Product Attention

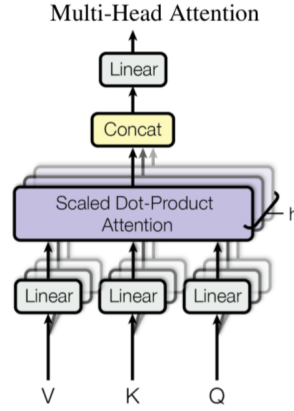


Figure 3.7 Multi-head Attention Consist of Several Attention Layers Running in Parallel

Transformer was shown in Figure 3.8. Inspired by their research, we leveraged Vision Transformer part to process the sequential features vectors extracted from CNN or MLP, and the architectures will be explained in next section (3.3.5).

3.3.5 Hybrid Deep Learning Architecture

3.3.5.1 CNN + Transformer Encoder

In this model, we combined a CNN architecture that consist of two convolutional layers with six layers transformer encoder. A CNN architecture was leveraged to segment the window data to

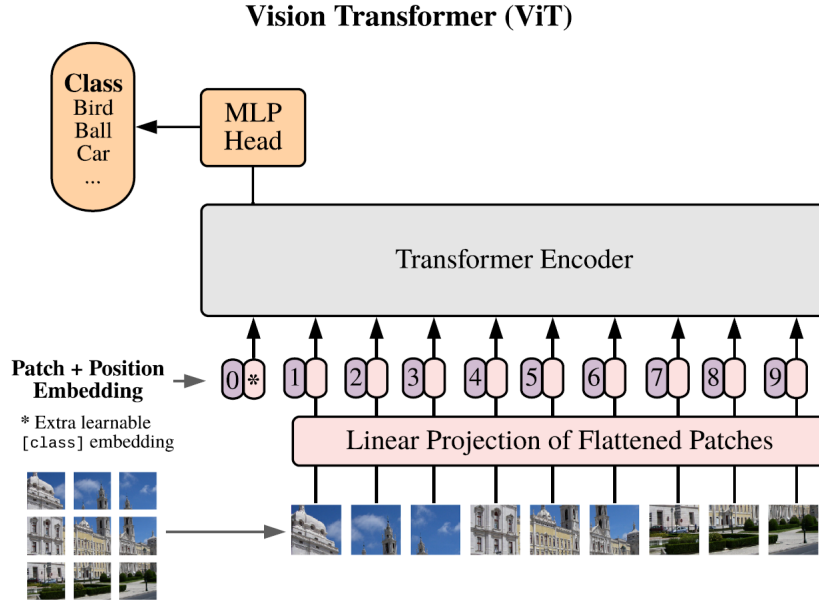


Figure 3.8 The Architecture of the Vision Transformer

24 3×3 patches with a 3×3 filter. The feature vectors output from CNN layers were then combined with position embedding vectors and fed into six Transformer encoders. The Transformer architecture we applied was the same with Vision Transformer (ViT) encoders. The overview of the architecture is shown in Figure 3.9.

3.3.5.2 Multi-MLP + Transformer Encoder

In this hybrid model, the data from each frame was flattened and processed by a MLP with two fully connected layers. In a window containing 60 frames, each feature vector output from MLP were combined with a position embedding vector and then fed into six Transformer encoders in parallel. See the architecture of the model in Figure 3.10.

3.3.5.3 Multi-CNN + Transformer Encoder

In this hybrid model, the data from each frame were processed by a CNN with two convolutional layers and a 3×3 filter. In a window containing 60 frames, each feature vector

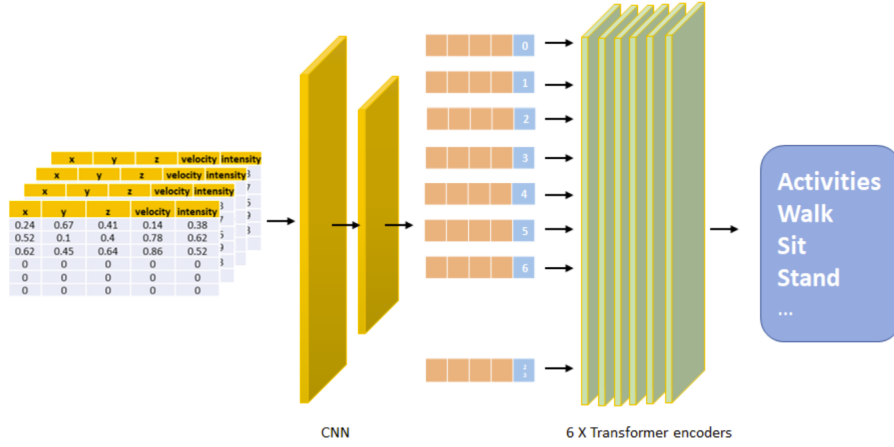


Figure 3.9 The Architecture of CNN and the Transformer Hybrid Model

output from a CNN were combined with a position embedding vector and then fed into six Transformer encoders in parallel. The architecture of the model is illustrated in Figure 3.11.

3.3.5.4 Training and Prediction

Total time of the radar sensor data is 93 minutes, including 71.6 minutes of training files and 21.4 minutes of test files. After data preprocessing, we obtained 12150 windows of data from training files, and 3565 windows of data from test files. The 12150 training data samples were then split into training dataset and validation dataset with a ratio of 8:2. To conclude, training dataset contains 9720 data samples, which are used for training model parameters like weights and biases. Validation dataset containing 2430 data samples are used for tuning parameters, while test dataset with 3565 data samples provides an unbiased evaluation of a final model.

Six models include three traditional deep learning models and three hybrid Transformer models were trained and tested on a node with 64 CPU, 64G memory and 1 GPU computational capacity on Pronto, a HPC service provided by Iowa State University.

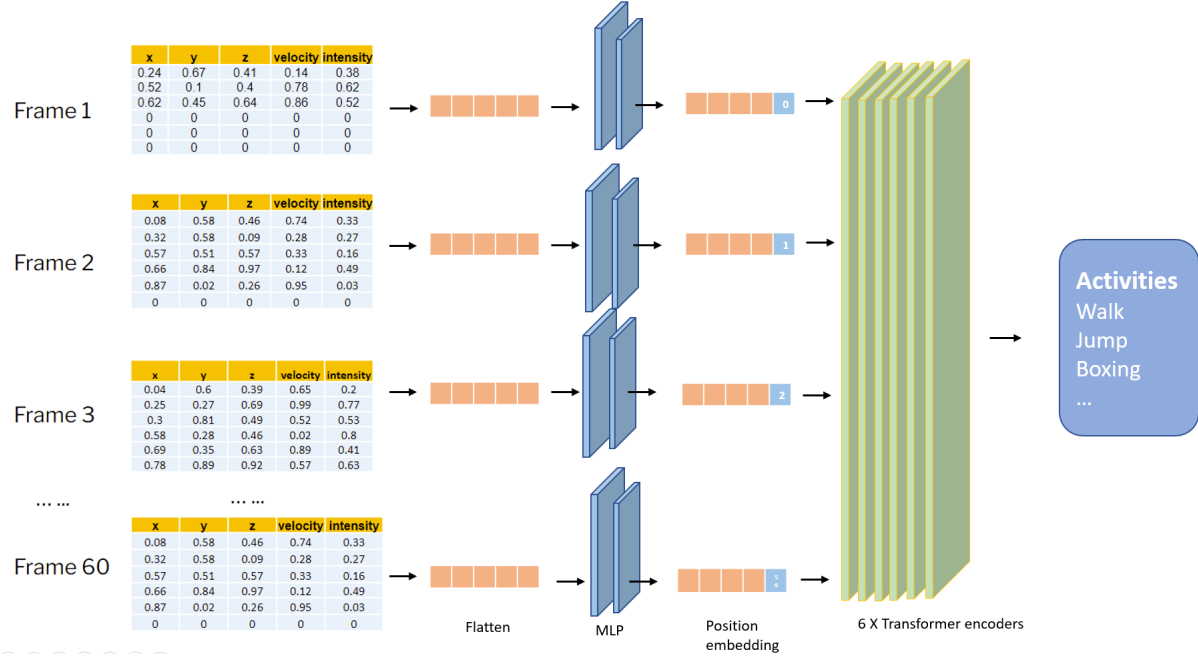


Figure 3.10 The Architecture of Multiple-MLP and the Transformer Hybrid Model

After training, the architectures and learnable parameters of the six models were saved. Furthermore, the human activities were predicted using each model with the test dataset, and test accuracies were evaluated and compared.

3.4 Equipment Setup and Data Collection

3.4.1 Equipment

The equipment we used to collect our point cloud dataset is a commercial mmWave Radar sensor from the Texas Instrument. The model name is IWR8643BOOST. Figure 3.12 which works at a frequency range from 76-GHz to 81-GHz. The sensor has on-board antenna hence is easy for indoor deployment. The approximate effective range for the sensor is from 6 to 10 meters which can cover a large area for a room.

The sensor transmits millimeter waves, and receives the reflections. The sensor supports two kinds of output format.

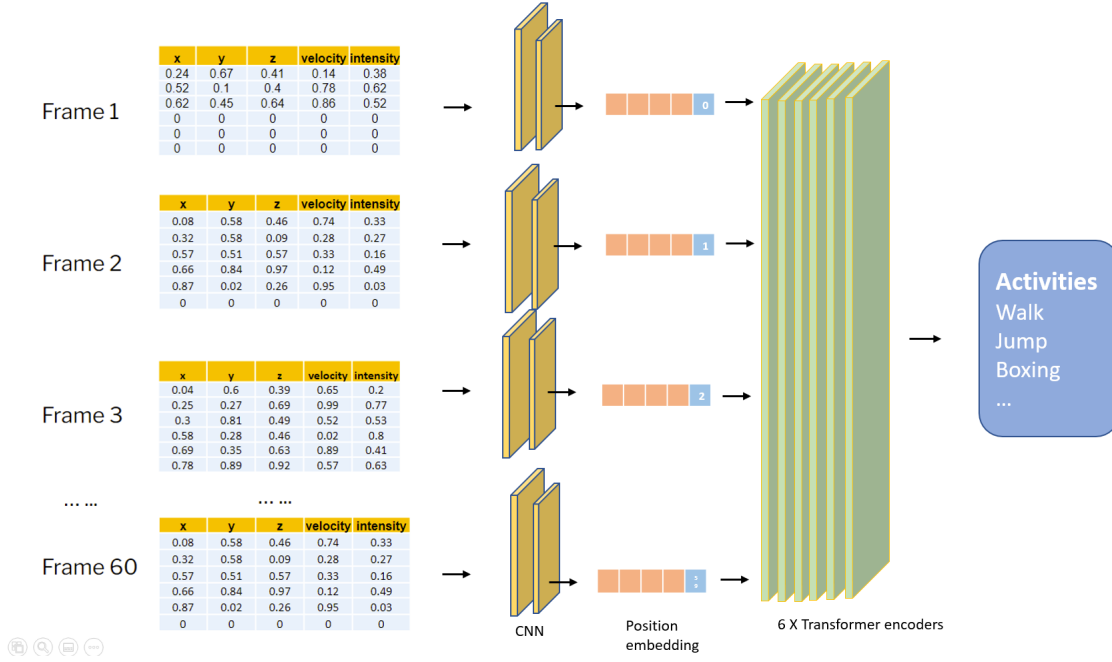


Figure 3.11 The Architecture of Multiple-CNN and the Transformer Hybrid Model

1. Raw data: As shown in Figure 3.13. Very much like a microphone which captures the sound signal, the raw data produced by the sensor is the mmWave signal which shows the amplitude along the time captured by each channel. The sample rate of the device is 625 ksp/s.
2. 3D point cloud: the point cloud data can be obtained using the onboard DSP and MCU evaluation module. As can be seen from Figure 3.14, in the data (x, y, z) coordinates of points represents the objects detected. In addition, the each point also have two extra attributes: velocity and intensity which often give information about the object's type. The frame rate given by the device output is 30 fps.

The raw data produces larger amount of data per second and contains much more noise.

Thus, we decided to use the 3D point cloud data for our system.

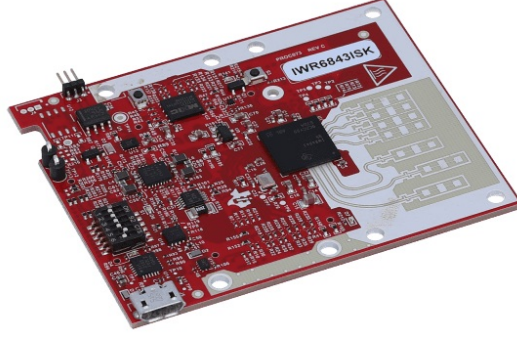


Figure 3.12 IWR8643 Packaged mmWave Sensor with DSP and MCU Evaluation Module

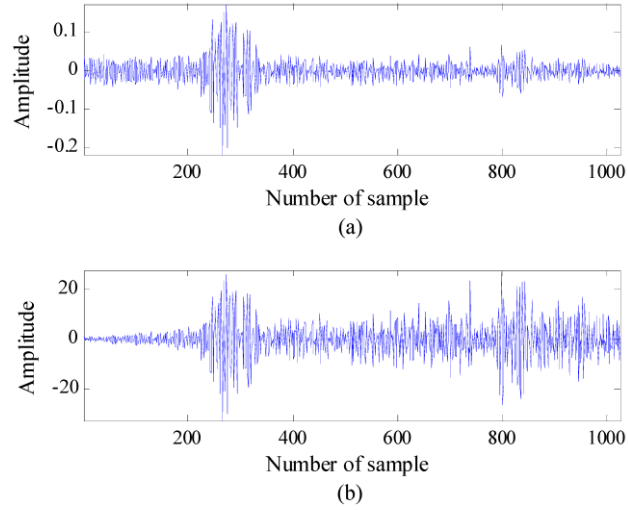


Figure 3.13 An Example of Raw Data

3.4.2 Data collection

Data was gathered from two subjects. The subjects performed five different activities which were walking, workout, sits, stands and falls in front of the radar. Figure 3.15 shows the examples of these five activities we collected. The data of walking and workout was collected in continuous periods of 5 minutes respectively. To grab the transitions from sit to stand and stand to sit, we asked the subjects to sit and stand 5 seconds alternatively for a total period of 10 minutes. To collect the falls activity, we let the subjects fell on the mat every 10 seconds in a total period of

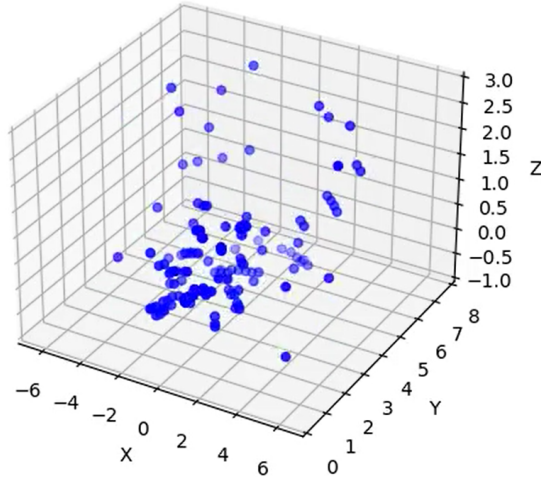


Figure 3.14 An Example of 3D Point Clouds

10 minutes. The point cloud data was saved in csv files with a timestamp for each frame. We also saved the screen recordings which showed the videos from a RGB camera and a depth camera.

3.5 Automatic Labeling

To train the deep learning model, annotation of the point clouds radar data to different human activities is an important step. In our case, we need to find the point cloud frames which represent a certain human activity and tag them. For activities like walking and workout, since they are both continuous movements, we only need to remove the unrelated frames at the beginning and the end of the data, and label the rest of the frames with the activity respectively. However, falls, sits and stands are not continuous, so fine labeling is needed. For instance, we only want to tag the period that the person was falling, but not the periods that he was laying the mat or getting up. Moreover, sits and stands were mixed up in the same file and we want to separate these two activities. We could check with the RGB video and manually label the corresponding point clouds data frames. However, it will not be feasible to manually label a large amount of data. Furthermore, the frames per second (fps) of RGB video and point clouds data are not the

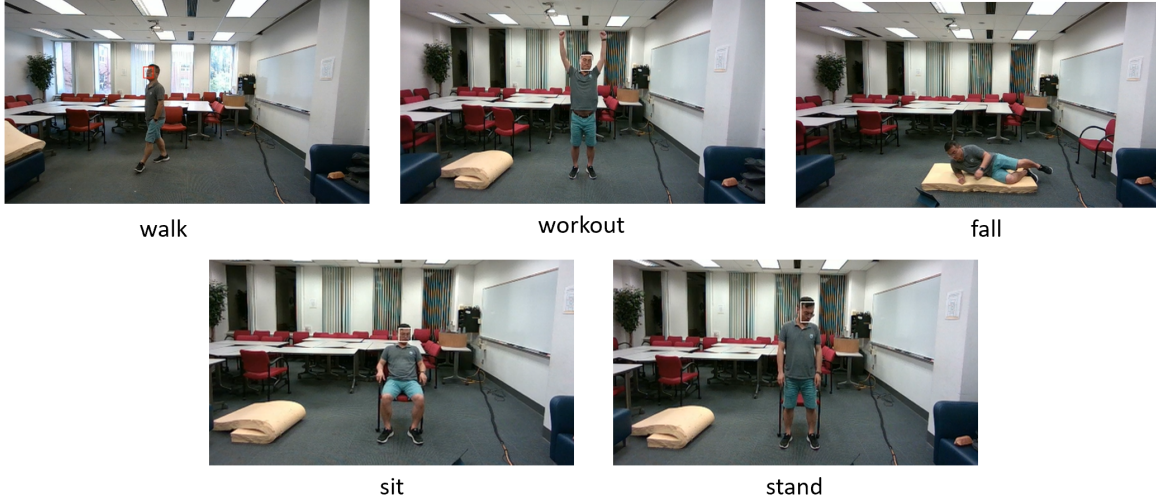


Figure 3.15 Five Collected Activities

matched which made manually labeling time consuming and error prone. Therefore, we proposed an automatic labeling for our point clouds radar dataset.

When we aggregated the non-static points within each data frame of the activities, plotted it with the timestamps, and checked with the corresponding RGB frame, we could clearly see the plot followed a rule: When the subjects moved, the non-static points increased, and when the subjects stayed on the mat or sit on the chair, non-static points decreased. As seen in Figure 3.16, the red dash boxes represent the period that the subject was laying on the mat (top figure) or sitting on the chair (bottom figure).

The idea of our automatic labeling algorithm is to first set up an threshold of number of points (Figure 3.17 (a)), and then aggregate the contiguous points under the threshold ((Figure 3.17 (b)). In the end, setting another threshold to get start points of the fall or sit activities.

Once we got a timestamp of the a start point of sitting, e.g. t_s , we labeled the time period between $(t_s - 5)$ and t_s as “sits”, and time period between $(t_s - 10)$ and $(t_s - 5)$ as “stands”. To label the falls activity, let us say we got the point of completely fall on the floor t_f , then we

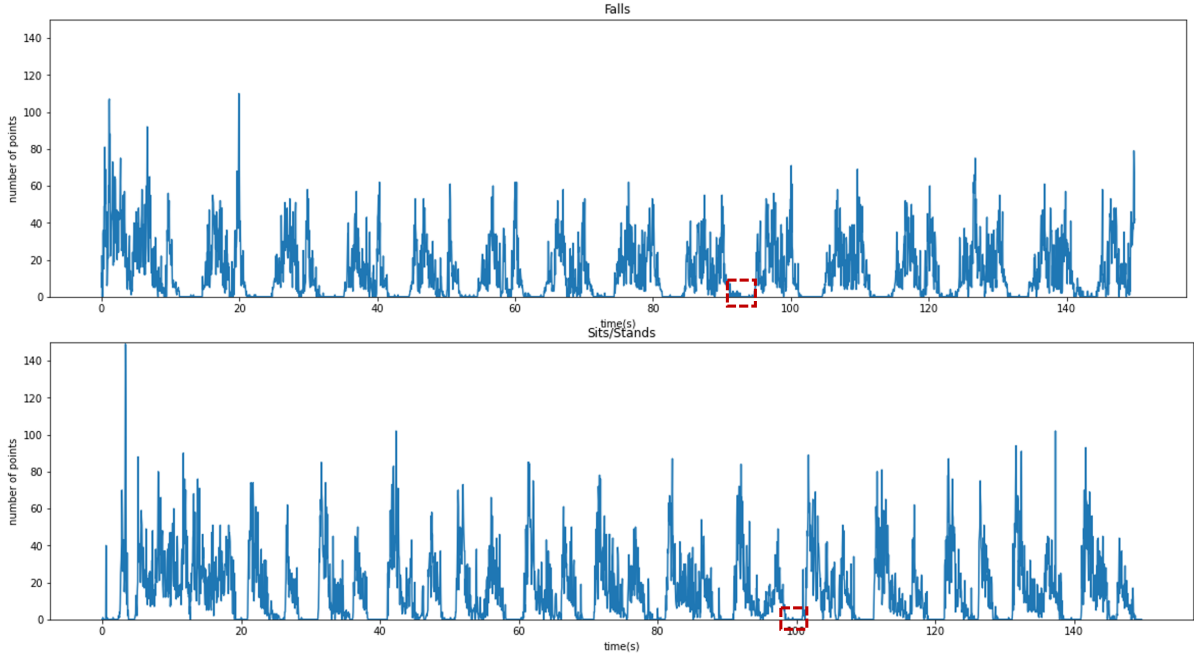


Figure 3.16 Plots of Number of Non-static Points of Falls and Sits/Stands activities

marked the time period between $((t_f - 1)$ and t_f as “falls”, since it took approximately 1 second for a subject to fall down based on the corresponding RGB video.

3.6 Data Preprocessing, Training and Prediction

After automatic labeling, we leveraged the same data preprocessing method that used for MMAActivity dataset. All the hyperparameters were kept the same, except for the sliding window size. For MMAActivity dataset, the size of sliding window was 60 frames, but we adjusted the sliding window size to 30 frames due to the smaller dataset we collected. After data preprocessing, we got total 3374 samples with 1171 samples of walking, 1231 samples of workout, 372 samples of sits, 372 samples of stands and 228 samples of falls. Data was split into train and validation dataset by a ratio 8:2.

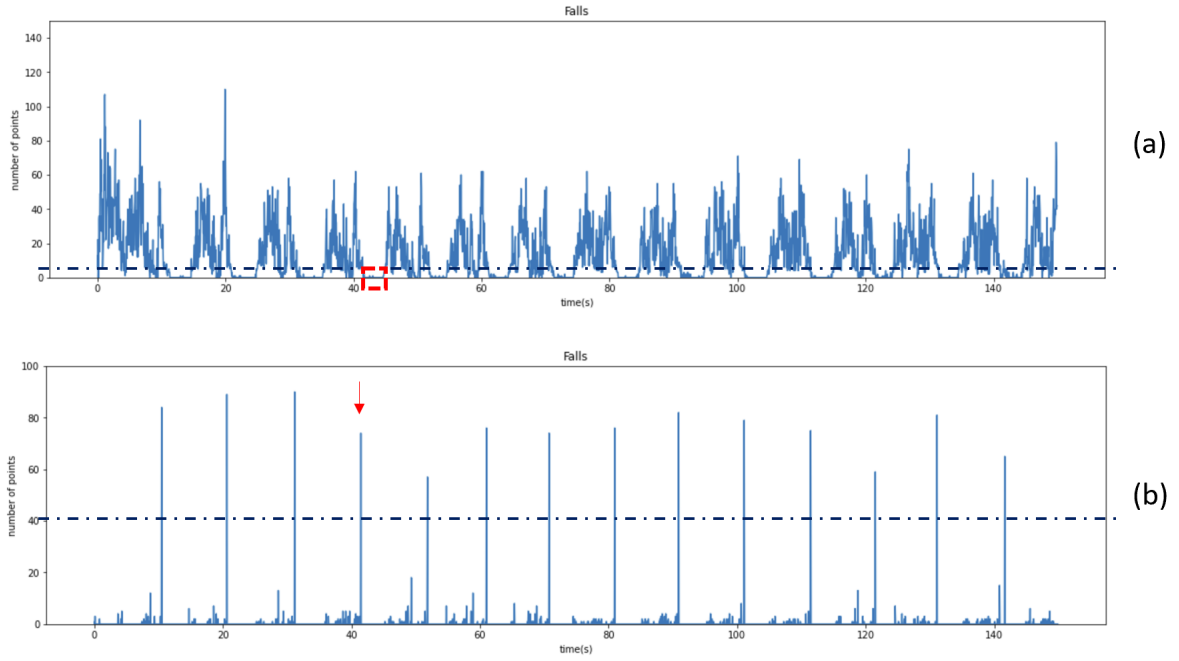


Figure 3.17 An Overview of Automatic Labelling

The model architectures we used for our data were tuned CNN+Transformer and BiLSTM model we tested in MMActivity dataset. We adopted the tuned hyperparameters from the CNN+Transformer model to our dataset.

After training, the validation dataset was utilized as the test dataset to evaluate the models.

CHAPTER 4. RESULTS

4.1 Model Comparison

Six deep learning models including CNN, LSTM, BiLSTM and three hybrid models were built and trained and validated with MMAActivity dataset. After trained, the model was evaluated using the test dataset split from MMAActivity dataset.

During each epoch in the training process, training loss and validation loss were measured and a loss graph were drawn from each model. The loss graph is an important graph to evaluate the performance of a model. In a good fit model, the training loss and validation loss are close to each other with validation loss being slightly greater than the training loss. If the training loss and validation loss are far away from each other, it means the model is overfitting which will not be generalize. Figure 4.1 shows the losses graphs of all six models. From the graph, we can see LSTM and BiLSTM performed better than other models. Among the three hybrid Transformer models, the model CNN+Transformer had the best performance.

The validation accuracy which is also an important metric to evaluate the performance of the model was collected during the train process from each model, and compared among all models. The test accuracy which indicates how well the trained model can predict on the a test dataset was also computed and compared. Table 4.1 shows the comparisons of validation accuracies, test accuracies and time cost of these six deep learning models. The result also indicated LSTM and BiLSTM performed best and the model CNN+Transformer outperformed the other two Transformer models.

A confusion matrix is a table to summarize the prediction results on a classification problem. The confusion matrix can show which classes the model is confused with when it makes predictions. Figure 4.2 shows the confusion matrices of the six neural network models. From the confusion matrices, we can see the models confused most is jumping and walking activities which

may imply the movement of jumping was too smaller to make the model to detect. It also shows that models are confused with boxing and jumping jack suggesting the radar sensor may be insensitively to capture the arms movements.

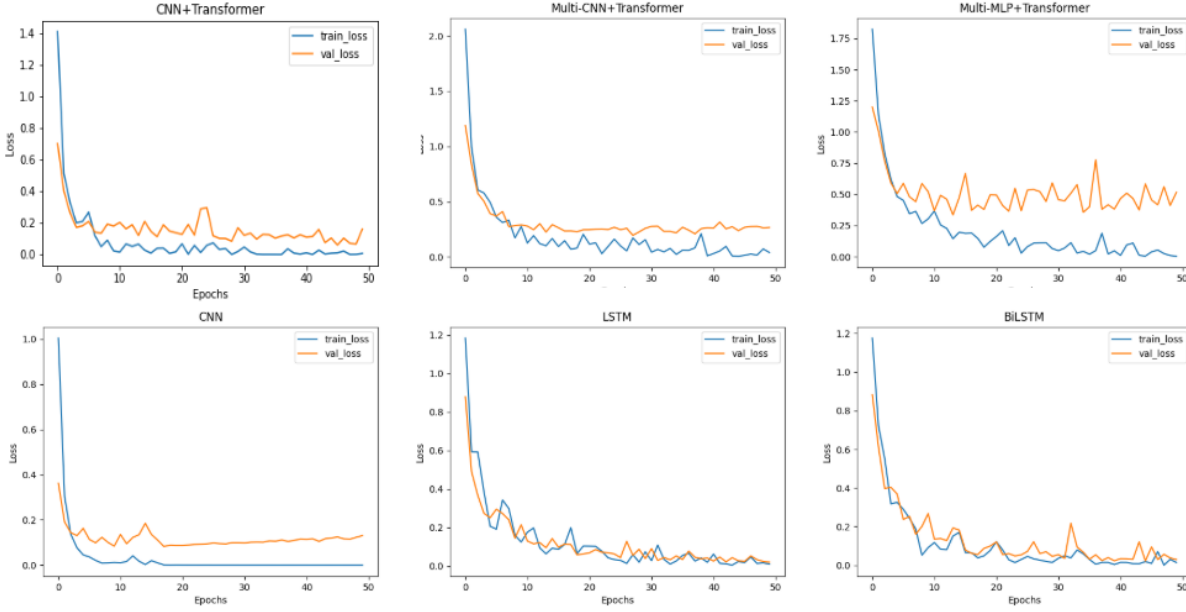


Figure 4.1 Loss Graphs of Models

Table 4.1 Model Comparison

Model	Validation Accuracy	Test Accuracy	Training Speed (s/epoch)
CNN+Transformer	95.72%	91.50%	5.97
Multi-MLP + Transformer	88.89%	85.67%	15.93
Multi-CNN +Transformer	91.52%	86.17%	18.42
CNN	95.80%	93.77%	0.95
LSTM	96.83%	95.18%	3.58
BiLSTM	97.04%	95.90%	3.51

4.2 Hyperparameter Optimization

Hyperparameters are a type of parameters which is used to control the model learning process. Unlike the model parameters which are trained from the data, hyperparameters are

manually defined, and affect the model performance. Hyperparameter optimization, also called hyperparameter tuning can improve the model's performance by adjusting the values of hyperparameters through multiple trials. In our research, since the CNN+Transformer model performed the best among the other Transformer models, we chose this model for further optimization.

Table 4.2 List of Tuned Hyperparameters

Hyperparameters	Descriptions	Tested Values
Depth of Transformer	The number of the Transformer encoders	[4,6,8,10,12,14,16,18]
Patch Size	Size of patches to segment the tabular data of each window	[1, 2, 3, 6]
Head Number	The number of heads for multi head attention	[4, 6, 10, 15]
Learning Rate	The learning rate to train a model	[0.05, 0.01, 0.005, 0.001, 0.0005, 0.0001]
Drop Out Rate	The rate of drop out layer within each of the Transformer encoder	[0.1, 0.2, 0.3, 0.4, 0.5]
Embedding Drop Out Rate	The rate of drop out layer after embedding layer	[0.1, 0.2, 0.3, 0.4, 0.5]

4.2.1 Hyperparameters Selection and Tuning

Table 4.2 shows the list of hyperparameters and values of them that are used to tune the model. Only one hyperparameter was modified in each trial, and then a loss graph was drawn from each hyperparameter in order to find the best value for the hyperparameter.

Figure 4.3 shows the loss curves of the trials in each parameter. The criteria for choosing a suitable value from a certain hyperparameter is to select the one with less loss value and more stable performance. As a result, we chose these values of the hyperparameters: depth = 6, patch = 6, head = 6, lr = 0.001, drop out rate = 0.5, embedding drop out rate = 0.1 for our final CNN+Transformer model. The CNN+Transformer model was rebuilt with these tuned hyperparameters.

4.2.2 Performance of Tuned Model

We can observe from Figure 4.4(b) that training loss and validation loss of the tuned model are closer than the untuned model Figure 4.4(a), which indicates the model has been improved. In addition, the accuracy and loss were also improved in the tuned model, see Figure 4.5.

The performance of tuned CNN+Transformer model were compared with other models as well. The validation accuracy was increased by 1%, and test accuracy was increased by 5% as shown in Table 4.3 indicating the model is much generalized than the original one. Moreover, we see the accuracy of the tuned model was as good as LSTM and BiLSTM which implies the CNN+Transformer model could replace the traditional deep learning models after further improvement in the future.

Table 4.3 Model Comparison with Tuned Model

Model	Validation Accuracy	Test Accuracy	Training Speed (s/epoch)
CNN+Transformer (Tuned)	96.38%	96.1%	4.83
CNN+Transformer (Untuned)	95.72%	91.50%	5.97
Multi-MLP + Transformer	88.89%	85.67%	15.93
Multi-CNN +Transformer	91.52%	86.17%	18.42
CNN	95.80%	93.77%	0.95
LSTM	96.83%	95.18%	3.58
BiLSTM	97.04%	95.90%	3.51

4.3 Comparison of Data Preprocessing Methods: Voxelization and Tabularization

Point clouds are disordered and uneven, thus they need to be preprocessed to be able to feed into deep learning models. Voxelization is a common data preprocessing methods for 3D point clouds data. Singh et al. (2019) used the same MMActivity dataset, but with voxelization preprocessing algorithm. They converted the point clouds data into voxels of dimension 10 x 32 x 32 with each voxel represents the number of data points within its boundaries. In our research, we converted the point clouds data to tabular format, and kept the data spatial information by data sorting. Compared with voxelization method, we found using this tabularization method greatly reduced data storage space, leveraged more features such as intensity and velocity, trained fast, and achieved a higher accuracy, as shown in Table 4.4. Our tabularization data preprocessing algorithm worked very efficient on processing radar point clouds data, as well as

reducing model training and model inference time, thus it could facilitate the application of HAR in industry in the near future.

Table 4.4 Comparison of Tabularization and Voxelization

	Tabularization	Voxelization
Data Size after Preprocessing	1.77 GB	71.9 GB
Features Leveraged	x , y, z, intensity, velocity	x, y, z
Test Accuracy (BiLSTM)	95.9%	88.42%
Training Speed (s/epoch)	3.51	15.52

4.4 Preliminary Results on the Collected Data from Smart Home Lab

To prove our pipeline of deep learning on human activities recognition and test our model architectures, we experimented our pipeline and optimized models on our own collected mmWave dataset. From the loss graphs of the two models, as seen in Figure 4.6, the training and validation losses were reduced as the training went on which indicated our pipeline and models worked with our collected dataset. We obtained an accuracy of 99.6% of BiLSTM model on 675 testing samples, while an accuracy of 93.6% of CNN+Transformer model on the same dataset. From confusion matrices of these two models, as seen in Figure 4.7, the CNN+Transformer model confused more on sits and stands activities, while BiLSTM is able to classify all the activities with high confidences.

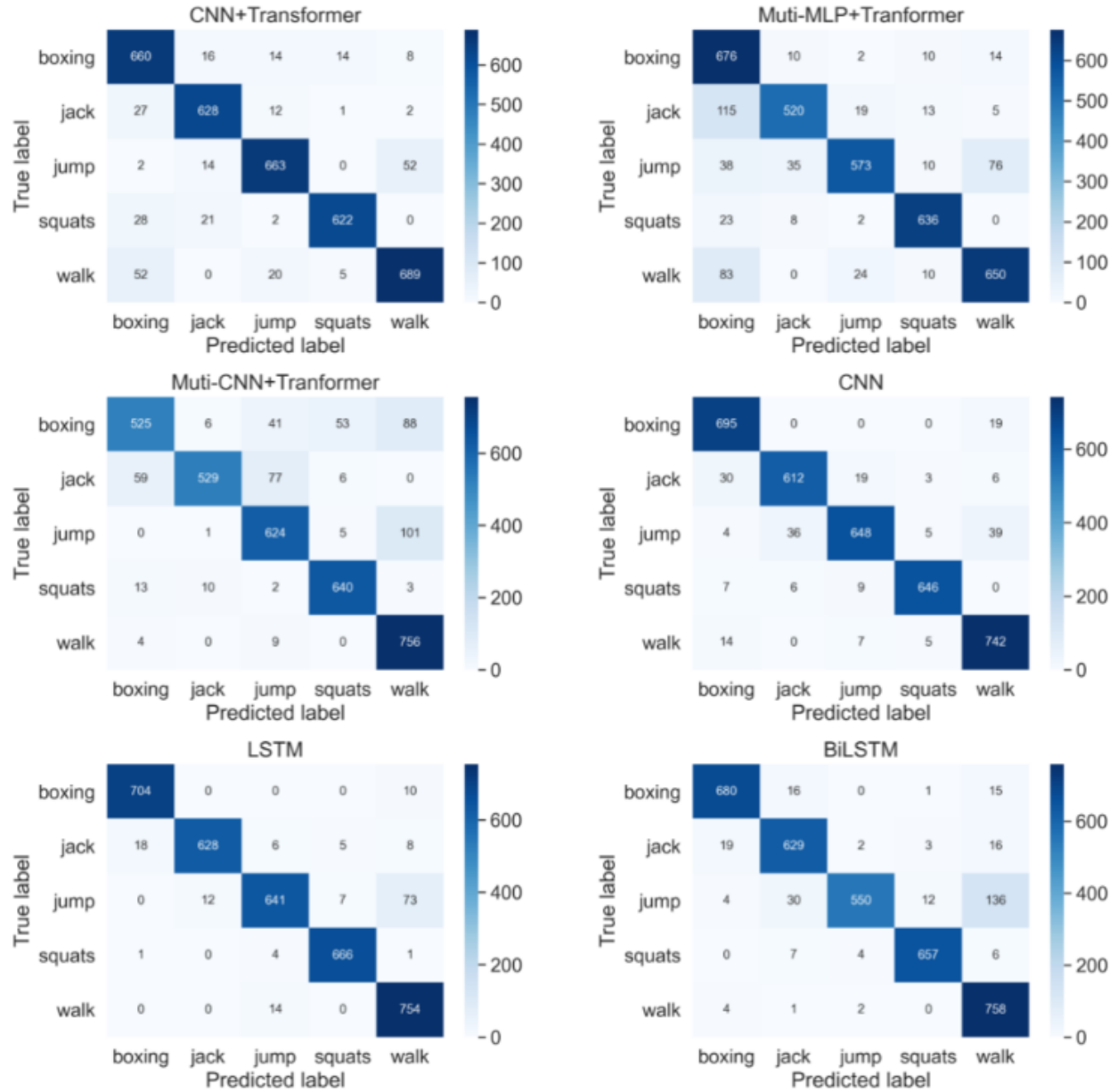


Figure 4.2 Confusion Matrices

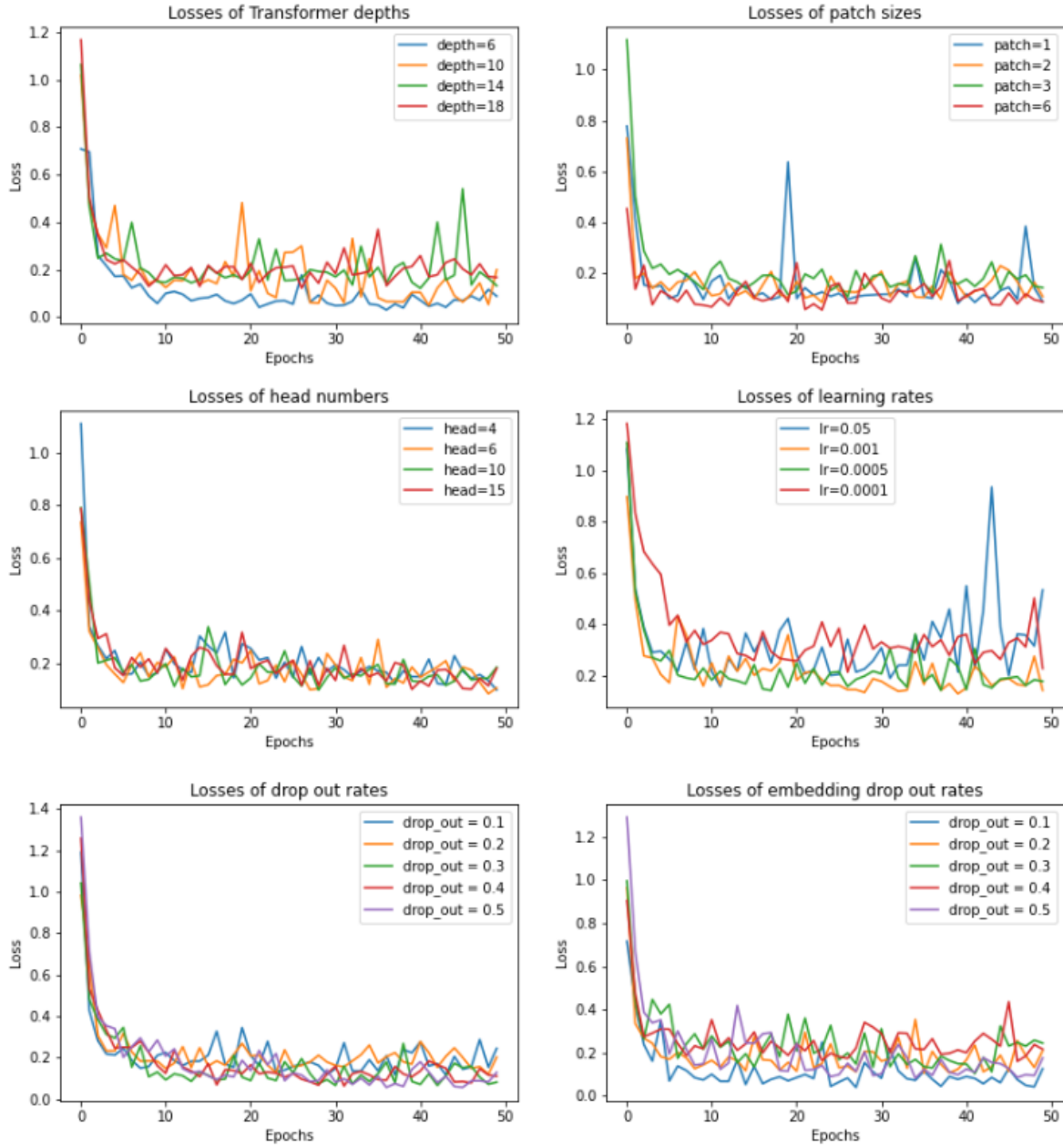


Figure 4.3 Loss Graphs of Different Hyperparameters during Tuning

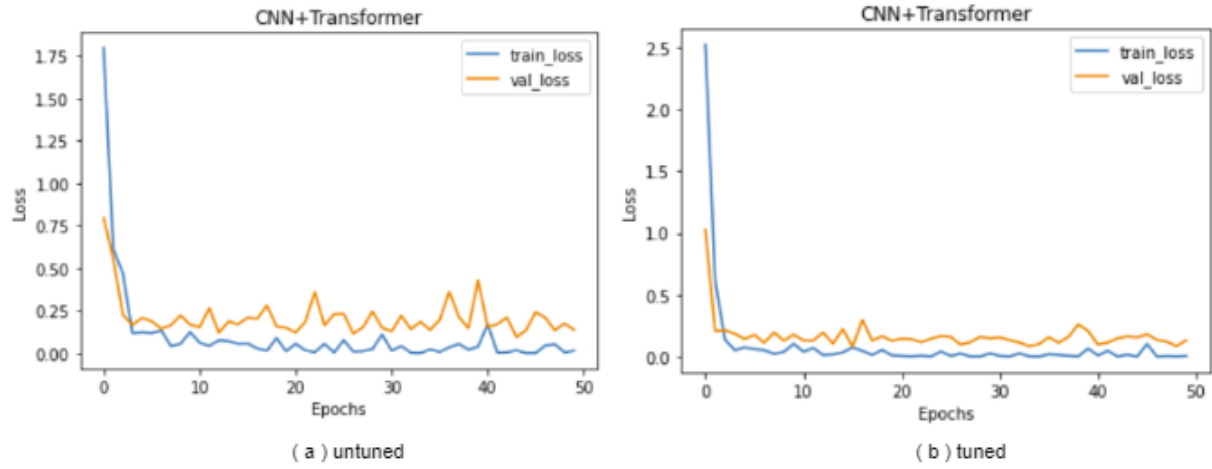


Figure 4.4 Comparison of Untuned Model and Tuned Model

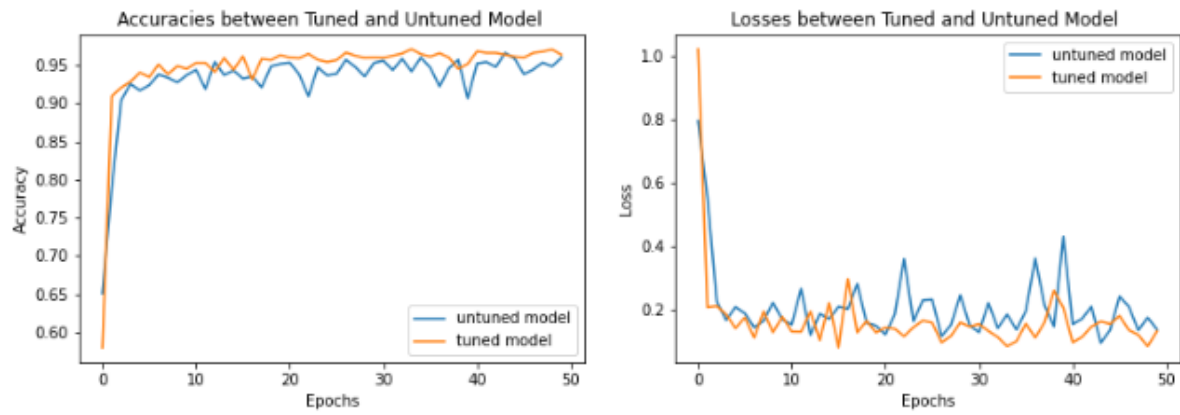


Figure 4.5 Accuracy and Loss Comparison of Untuned Model and Tuned Model

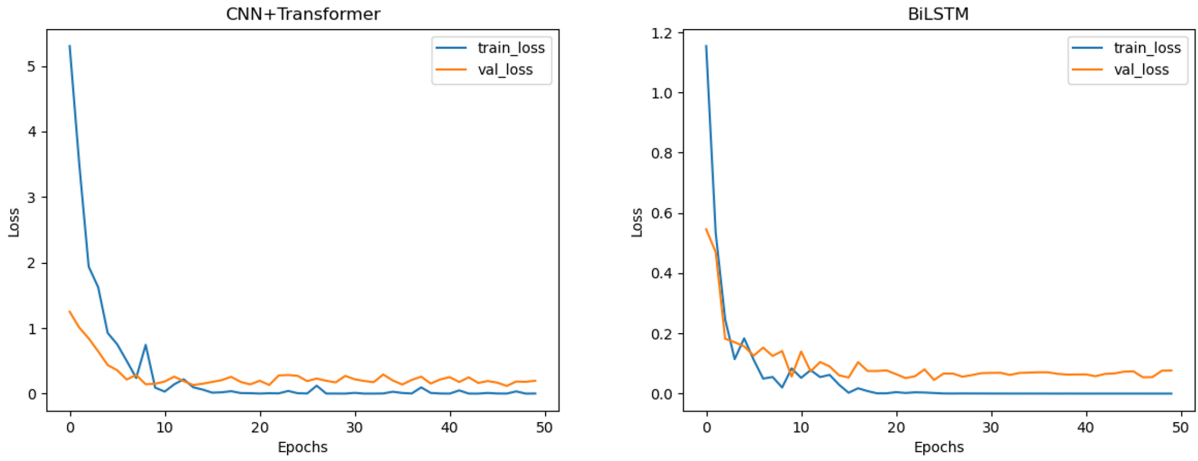


Figure 4.6 Loss Graphs of CNN+Transformer and BiLSTM Models with Smart Home Lab Data

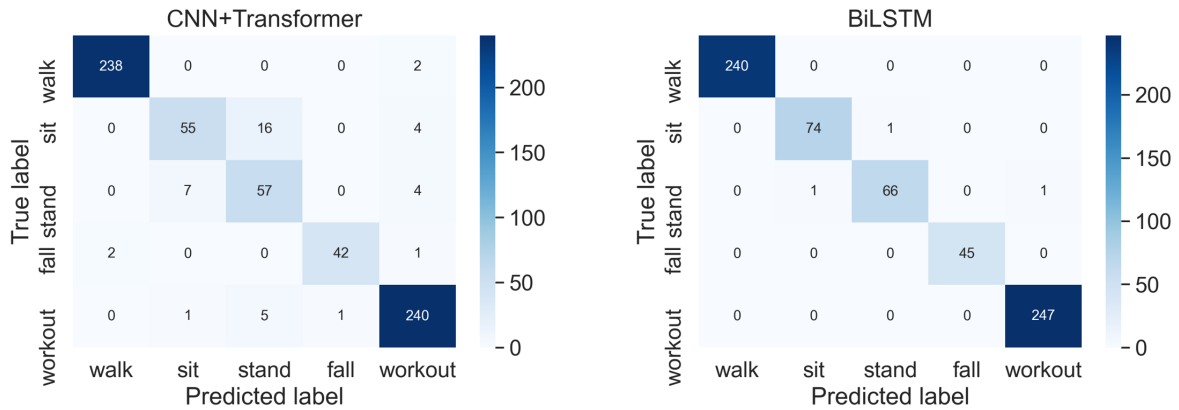


Figure 4.7 Confusion Matrices of CNN+Transformer and BiLSTM with Smart Home Lab Data

CHAPTER 5. SUMMARY AND DISCUSSION

In this report, we have developed a tabularization data preprocessing method, and proposed three Transformer deep learning algorithms to recognize human activities using a radar point clouds HAR dataset called MMAActivity. We also collected our own mmWave Radar dataset of five human activities, and applied our developed pipeline on it. The results demonstrated that the tabularization preprocessing method outperformed the traditional voxelization method in preprocessing radar point clouds data, and the CNN with the Transformer encoder architecture worked equally good as the traditional deep learning models in recognize human activities using the MMAActivity dataset. Moreover, we showed the preprocessing method and the Transformation algorithm worked well with our own collected mmWave cloud points radar dataset.

The main conclusions of this report are:

1. Proposed a tabularization data preprocessing method with data sorting, padding and sliding windows. By comparing with voxelization, tabularization algorithm generated less data size, thus decreased the training time while obtaining a high testing accuracy than using voxelization method.
2. Proposed three hybrid Transformer deep learning models to predict human activities. Compared the performances of the proposed Transformer models with three traditional deep learning models (CNN, LSTM, BiLSTM). With untuned models, it showed that LSTM and BiLSTM outperformed other deep learning models. Within the three hybrid Transformer models, the hybrid CNN and Transformer encoder architecture outperformed the other two.
3. Hyperparameter optimization was conducted within the best performed Transformer model (CNN+Transformer). Six hyperparameters were selected to be tested, and a set of values was experimented within each of the hyperparameters. After multiple trials, the values of the hyperparameters were determined and applied in the CNN+Transformer model.

4. After hyperparameter optimization, the performance of the CNN+Transformer model was greatly improved. The results indicated the model was less overfitting, and obtained a high accuracy as good as LSTM and BiLSTM using MMAActivity dataset.
5. Automatically labelling was designed and applied on our own point clouds human activity recognition dataset using mmWave radar sensor. Our established tabularization data reprocessing with data sorting, padding and sliding windows algorithms were applied on the dataset. CNN+Transformer and BiLSTM deep learning models with the same hyperparameters were experimented. We finally obtained an accuracy of 99.6% of BiLSTM model on 675 testing samples, while an accuracy of 93.6% of the CNN+Transformer model on the same dataset.

This study has proved that the pipeline of data tabularization preprocessing method and the Transformer deep learning architecture can be applied to recognize human activities, especially with our own collected mmWave dataset. Although the Transformer model performed equally well as BiLSTM with MMAActivity dataset, it worked not as well as BiLSTM with our own collected dataset. The size of the dataset played a very important role regarding the performance of the Transformer models. [Dosovitskiy et al. \(2020\)](#) showed that the Transformer models outperformed state of the art deep learning models when trained on large datasets. After data preprocessing we only obtained 2699 samples for training, which was much fewer than the number of training samples (9720) of MMAActivity dataset. In the future, we will increase our point clouds mmWave dataset by increasing longer activity time, adding diverse human activities, and including more subjects performing those activities. Meanwhile, we will conduct data fusion techniques by combining the point cloud data with the data collected from depth camera to obtain a higher accuracy on human activity recognition. Once the models are trained, and hyperparameters are optimized, we will deploy our model to the cloud-based server, such as Amazon AWS, as a part of human activity monitor system to serve for the health care areas such as the senior care industry.

BIBLIOGRAPHY

- Aljarrah, A. A. and Ali, A. H. (2019). Human activity recognition using pca and bilstm recurrent neural networks. In *2019 2nd International Conference on Engineering Technology and its Applications (IICETA)*, pages 156–160. IEEE.
- Banos, O., Galvez, J.-M., Damas, M., Pomares, H., and Rojas, I. (2014). Window size impact in human activity recognition. *Sensors*, 14(4):6474–6499.
- Bilkhu, M., Wang, S., and Dobhal, T. (2019). Attention is all you need for videos: Self-attention based video summarization using universal transformers. *arXiv preprint arXiv:1906.02792*.
- Çağhyan, B. and Gürbüz, S. Z. (2015). Micro-doppler-based human activity classification using the mote-scale bumblebee radar. *IEEE Geoscience and Remote Sensing Letters*, 12(10):2135–2139.
- Chen, K., Zhang, D., Yao, L., Guo, B., Yu, Z., and Liu, Y. (2020). Deep learning for sensor-based human activity recognition: overview, challenges and opportunities. *arXiv preprint arXiv:2001.07416*.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
- Fauzi, C., Sulisty, S., et al. (2018). A survey of group activity recognition in smart building. In *2018 International Conference on Signals and Systems (ICSigSys)*, pages 13–19. IEEE.
- Gavrilin, Y. and Khan, A. (2019). Across-sensor feature learning for energy-efficient activity recognition on mobile devices. In *2019 International Joint Conference on Neural Networks (IJCNN)*, pages 1–7. IEEE.
- Gers, F. A., Schraudolph, N. N., and Schmidhuber, J. (2002). Learning precise timing with lstm recurrent networks. *Journal of machine learning research*, 3(Aug):115–143.
- Girdhar, R., Carreira, J., Doersch, C., and Zisserman, A. (2019). Video action transformer network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 244–253.
- Gu, C., Sun, C., Ross, D. A., Vondrick, C., Pantofaru, C., Li, Y., Vijayanarasimhan, S., Toderici, G., Ricco, S., Sukthankar, R., et al. (2018). Ava: A video dataset of spatio-temporally localized atomic visual actions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6047–6056.

- Ha, S. and Choi, S. (2016). Convolutional neural networks for human activity recognition using multiple accelerometer and gyroscope sensors. In *2016 International Joint Conference on Neural Networks (IJCNN)*, pages 381–388. IEEE.
- Kim, Y. and Moon, T. (2015). Human detection and activity classification based on micro-doppler signatures using deep convolutional neural networks. *IEEE geoscience and remote sensing letters*, 13(1):8–12.
- Li, X., He, Y., and Jing, X. (2019). A survey of deep learning-based human activity recognition in radar. *Remote Sensing*, 11(9):1068.
- Myo, W. W., Wettayaprasit, W., and Aiyarak, P. (2019). Designing classifier for human activity recognition using artificial neural network. In *2019 IEEE 4th International Conference on Computer and Communication Systems (ICCCS)*, pages 81–85. IEEE.
- Parmar, N., Vaswani, A., Uszkoreit, J., Kaiser, L., Shazeer, N., Ku, A., and Tran, D. (2018). Image transformer. In *International Conference on Machine Learning*, pages 4055–4064. PMLR.
- Pienaar, S. W. and Malekian, R. (2019). Human activity recognition using lstm-rnn deep neural network architecture. In *2019 IEEE 2nd Wireless Africa Conference (WAC)*, pages 1–5. IEEE.
- Rosenblatt, F. (1961). Principles of neurodynamics. perceptrons and the theory of brain mechanisms. Technical report, Cornell Aeronautical Lab Inc Buffalo NY.
- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural networks*, 61:85–117.
- Singh, A. D., Sandha, S. S., Garcia, L., and Srivastava, M. (2019). Radhar: Human activity recognition from point clouds generated through a millimeter-wave radar. In *Proceedings of the 3rd ACM Workshop on Millimeter-wave Networks and Sensing Systems*, pages 51–56.
- Tealab, A. (2018). Time series forecasting using artificial neural networks methodologies: A systematic review. *Future Computing and Informatics Journal*, 3(2):334–340.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *arXiv preprint arXiv:1706.03762*.
- Wang, M., Zhang, Y. D., and Cui, G. (2019). Human motion recognition exploiting radar with stacked recurrent neural network. *Digital Signal Processing*, 87:125–131.
- Wang, S., Song, J., Lien, J., Poupyrev, I., and Hilliges, O. (2016). Interacting with soli: Exploring fine-grained dynamic gesture recognition in the radio-frequency spectrum. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, pages 851–860.

- Zhao, P., Lu, C. X., Wang, J., Chen, C., Wang, W., Trigoni, N., and Markham, A. (2019). mid: Tracking and identifying people with millimeter wave radar. In *2019 15th International Conference on Distributed Computing in Sensor Systems (DCOSS)*, pages 33–40. IEEE.